

Multithreading in C# with Examples

In this article, I will discuss **Multithreading in C#** with Examples. Multithreading is one of the most important concepts in C# that you must understand as a developer. In this and a few upcoming articles, I will cover all the concepts of C# Multithreading with examples. As part of this article, I will cover the following pointers.

1. **What is Multitasking?**
2. **How Does the Operating System Execute Multiple Applications at a time?**
3. **What is a Process?**
4. **What is a Thread?**
5. **Example to Understand Threading in C#.**
6. **Understanding the Thread Class in .NET Framework.**
7. **What are the Drawbacks of Single-Threaded Applications?**
8. **How to Overcome the Drawbacks of Single-Threaded Applications Using Multithreading?**
9. **What is Multithreading in C#?**
10. **What is Thread Class in C#?**

What is Multitasking?

Before understanding the concept of Multithreading in C#, let us first understand multitasking. As the name says, multitasking means performing multiple tasks simultaneously. Windows Operating System is a multitasking operating system. It means it has the ability to run multiple applications at the same time. For example, on my machine, I can open the Google Chrome Browser, Microsoft Word Document, Notepad, VLC Media Player, Visual Studio, etc., at the same time. This is possible because, on my machine, I have installed the Windows operating system, which is a multitasking operating system.

So, Multitasking refers to the concurrent execution of multiple tasks or processes on a computer system. It allows a computer to appear as though it is performing multiple tasks simultaneously, even though it typically has a single central processing unit (CPU). Multitasking is a fundamental feature of modern operating systems and plays an important role in improving computers' efficiency, responsiveness, and usability.

How Does the Operating System Execute Multiple Applications at a time?

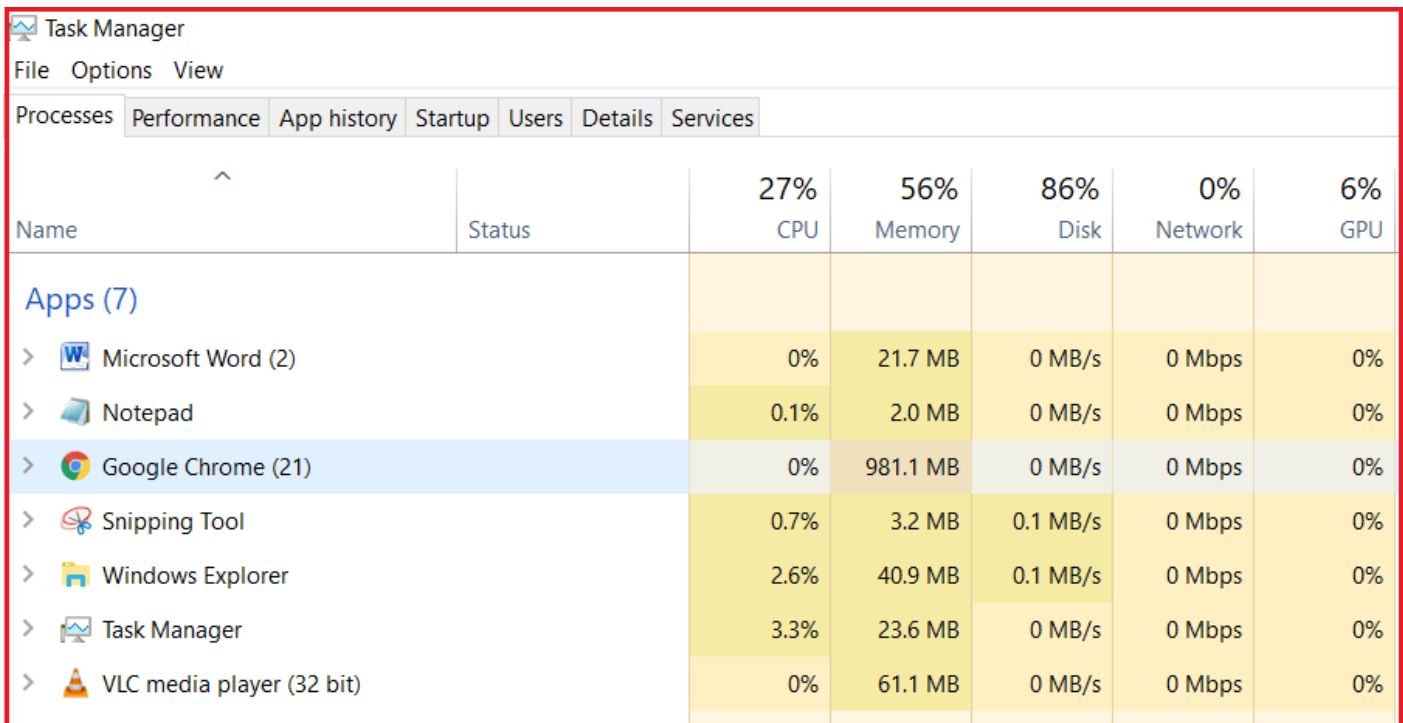
The operating system uses processes internally to execute multiple applications (Google Chrome Browser, Microsoft Word Document, Notepad, VLC Media Player, Visual Studio, etc.) simultaneously.

What is a Process?

A process is a part of the operating system (or a component under the operating system) responsible for executing the program or application. So, to execute each and every program or application, there will be a process.

In an operating system (OS), a process is a fundamental concept representing a program in execution. It is the smallest unit of work in a computer system's execution. Each process has its own memory space, which includes the program code, data, and a stack for managing function calls and local variables. The OS manages processes and has its own resources, such as CPU time, file handles, and system state information.

The processes executing the programs or applications using the Task Manager. Just right-click on the Taskbar and click the Task Manager option to open the Task Manager window. From that window, click on the “Processes” button as shown below.



Name	Status	27% CPU	56% Memory	86% Disk	0% Network	6% GPU
Apps (7)						
> Microsoft Word (2)		0%	21.7 MB	0 MB/s	0 Mbps	0%
> Notepad		0.1%	2.0 MB	0 MB/s	0 Mbps	0%
> Google Chrome (21)		0%	981.1 MB	0 MB/s	0 Mbps	0%
> Snipping Tool		0.7%	3.2 MB	0.1 MB/s	0 Mbps	0%
> Windows Explorer		2.6%	40.9 MB	0.1 MB/s	0 Mbps	0%
> Task Manager		3.3%	23.6 MB	0 MB/s	0 Mbps	0%
> VLC media player (32 bit)		0%	61.1 MB	0 MB/s	0 Mbps	0%

As you can see in the above image, each application is executed by one corresponding process. Along the same line, multiple processes are running in the background, known as the background processes. These background processes are known as Windows Services, and the Operating System runs many Windows services in the background.

So, we have an operating system, and under the operating system, we have processes that run our applications. So, the point that you need to remember is under the process, an application runs. To run the code of an application, the process internally uses a concept called Thread.

What is Thread?

Generally, a Thread is a lightweight process. In simple words, we can say that a Thread is a unit of a process that is responsible for executing the application code. So, every program or application has some logic or code, and to execute that logic or code, Thread comes into the picture.

In an operating system (OS), a thread is a smaller unit of execution within a process. Threads are sometimes called “lightweight processes” because they share the same memory space as the parent process, including its code, data, and resources. However, threads have their own execution context, including a program counter, registers, and stack. Threads within the same process can run concurrently, allowing for parallel execution of tasks.

By default, every process has at least one thread that is responsible for executing the application code, and that thread is called Main Thread. So, every application by default is a single-threaded application.

Note: All the threading-related classes in C# belong to **System.Threading** namespace.

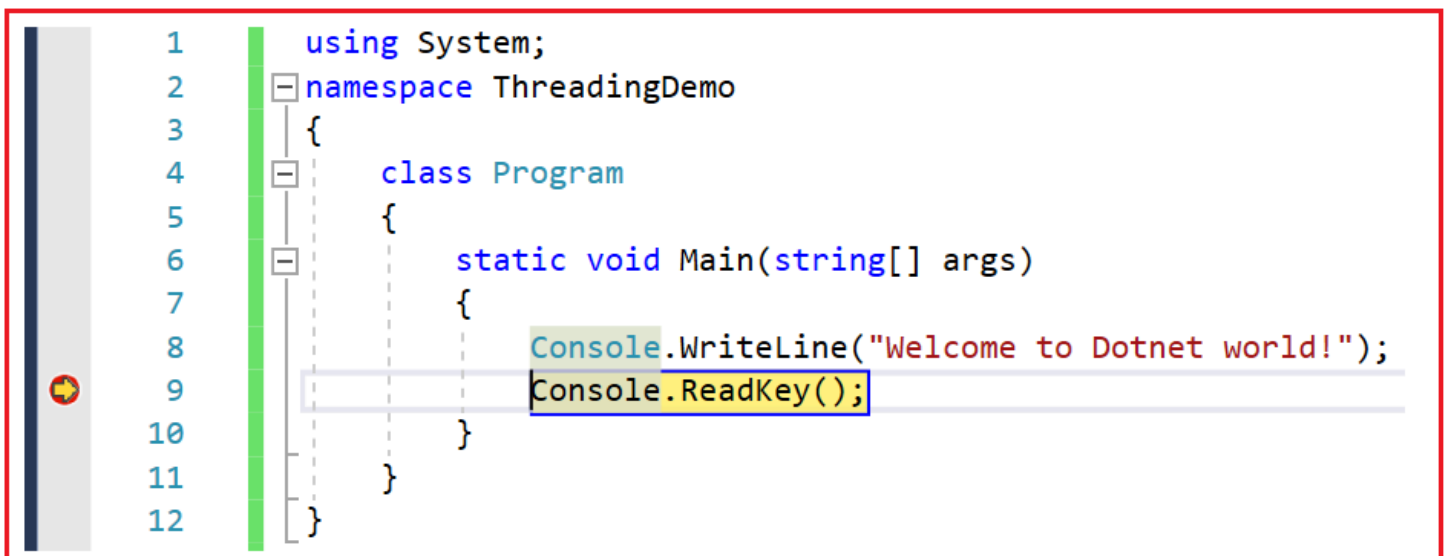
Example to Understand Threading in C#:

Let us see an example to understand Threading in C#. The following is a simple program where we have a class called Program, and in that class, we have a method called Main, which simply prints a message on the Console window.

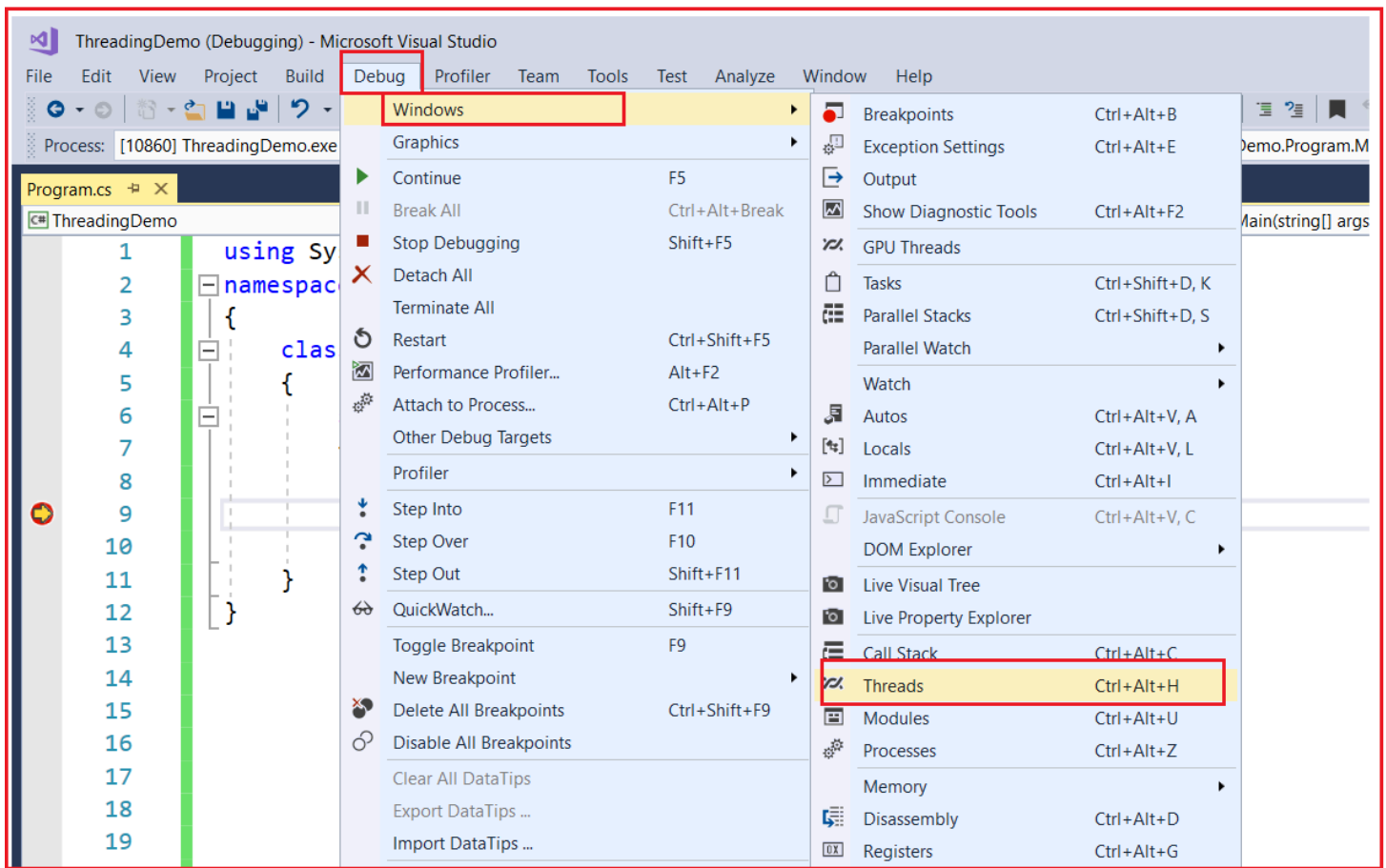
```
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Welcome to Dotnet world!");
            Console.ReadKey();
        }
    }
}
```

Does the above Program Make use of Thread When Run?

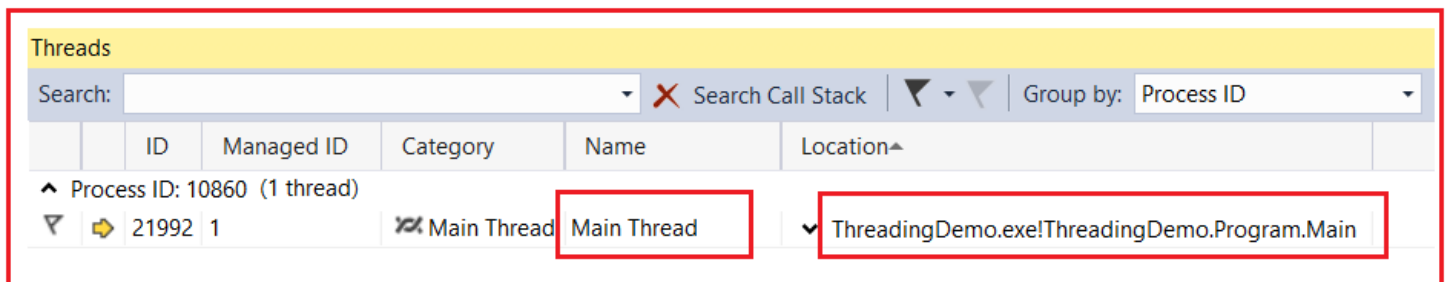
Yes, there is a Thread, and that thread will execute our application code, which is called the Main Thread. Now, let us prove this. Put a debugger point on your application code and then run the application. Let us assume you put the debugger point on line number 9, and when you run the application, you will see that the debugger will hit that point, as shown in the below image.



Once the debugger hits the debugging point, select the **Debug => Windows => Threads** options from the Visual Studio menu, as shown in the below image.



Once you select the **Debug => Windows => Threads** options, then it will open the following window. You can see here that the Main Thread is currently executing the Main method of the Program class.



So, this proves that by default, every application is a single-threaded application, and by default, the Main thread will execute our application code.

Thread Class in C#:

Go to the definition of Thread Class in C#. You will see that it contains one static property called **CurrentThread**, which will return the instance of the currently executing thread, i.e., the thread running your application code. If you go to the definition of Thread class, you will find the following signature.

```

public sealed class Thread : CriticalFinalizerObject, _Thread
{
    -- Other Methods and Properties
    //
    // Summary:
    // Gets the currently running thread.
    //
    // Returns:
    // A System.Threading.Thread that is the representation of the currently running thread.
    public static Thread CurrentThread { get; }
    public string Name { get; set; }
    -- Other Methods and Properties

    public static void Sleep(TimeSpan timeout);
    public static void Sleep(int millisecondsTimeout);
}

```

As you can see, the **CurrentThread** static property return type is Thread, i.e., it will return the instance of the currently executing thread. Along the same line, there is a non-static property called Name, using which we can set and get the Name of the currently executing thread.

Note: By default, the thread does not have any name. You can provide any name to the thread using the Name property of the Thread class. So, modify the program as shown below, where we set the Name property of the thread class and printing the Name property.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t = Thread.CurrentThread;
            //By Default, the Thread does not have any name
            //if you want then you can provide the name explicitly
            t.Name = "Main Thread";
            Console.WriteLine("Current Executing Thread Name :" + t.Name);
            Console.WriteLine("Current Executing Thread Name :" +
Thread.CurrentThread.Name);
            Console.Read();
        }
    }
}

```

Output:

```
Current Executing Thread Name :Main Thread  
Current Executing Thread Name :Main Thread
```

As you can see, to run the application code, one thread is created, i.e., the Main Thread. So, this proves that, by default, every application is a single-threaded application.

What are the drawbacks of Single-Threaded Applications in the .NET Framework?

Single-threaded applications in the .NET Framework have several drawbacks, particularly regarding performance, responsiveness, and scalability. Some of the main drawbacks include:

- **Limited CPU Utilization:** Single-threaded applications can only execute one task or operation simultaneously. This means they cannot fully utilize modern multi-core processors, leading to the underutilization of available CPU resources. In contrast, multi-threaded or parallel applications can distribute work across multiple threads and utilize more CPU cores efficiently.
- **Poor Responsiveness:** In a single-threaded application, time-consuming operations can block the entire execution, making the application appear unresponsive to user input. For example, if a long-running computation occurs on the main thread, the user interface won't update until the computation is complete. This can result in a poor user experience.
- **Difficulty Handling Concurrent Tasks:** Single-threaded applications struggle to handle concurrent tasks, such as responding to user input, while performing background tasks like data loading or computation. The application may become unresponsive during such operations without additional threads or asynchronous programming techniques.
- **Scalability Challenges:** Single-threaded applications often face challenges when scaling to simultaneously handle multiple users or workloads. They may not be well-suited for scenarios requiring high concurrency or parallel processing, such as web servers or real-time data processing applications.
- **Limited Parallelism:** Single-threaded applications cannot perform true parallel processing. This can be a significant drawback when dealing with computationally intensive tasks that can benefit from parallelism, such as data analysis, rendering, or simulations.
- **Inefficient Resource Utilization:** Single-threaded applications may not efficiently utilize other system resources, such as memory and I/O. For example, they might block the thread while waiting for I/O operations to complete, leading to wasted CPU cycles that could be used for other tasks.
- **Difficulty in Achieving Fault Tolerance:** In single-threaded applications, handling errors and exceptions can be more challenging because a failure in one part of the application can disrupt the entire process. Multi-threaded or multi-process architectures can isolate failures to specific threads or processes, making fault tolerance easier.

To address these limitations, developers often use multi-threading, asynchronous programming, or parallel processing techniques provided by the .NET Framework to make applications more responsive, scalable, and efficient. These approaches enable applications to utilize modern hardware better and improve user experiences, especially in concurrent tasks and heavy workload scenarios.

In a single-threaded application, all the logic or code present in the program will be executed by a single thread only, i.e., the Main thread. For example, if we have three methods in our application. All these three methods will be called from the Main method. Then, the Main thread is only responsible for executing all these three methods sequentially, i.e., one by one. It will execute the first method, and once it completes the execution of the first method, it only executes the second method, and so on. Let us understand this with an example. Modify the program as shown below.

```
using System;
```

```
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Method1();
            Method2();
            Method3();
            Console.Read();
        }
        static void Method1()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }

        static void Method2()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method2 :" + i);
            }
        }
        static void Method3()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method3 :" + i);
            }
        }
    }
}
```

Output:

```
Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method1 :5
Method2 :1
Method2 :2
Method2 :3
Method2 :4
Method2 :5
Method3 :1
Method3 :2
Method3 :3
Method3 :4
Method3 :5
```

As you can see in the above output, the methods are called and executed one after the other. The Main thread first executes Method1, and once it completes the execution of Method1, it calls Method2 and Method3.

What is the problem with the above Program Execution?

In our example, we are just writing some simple code to print the values from 1 to 5 within the method body. What will you do if one method takes more than expected time? Suppose Method2 is going to interact with a database, or it is going to invoke any web services or any web APIs, which will take more than 10 seconds to provide the response. In that case, the Method2 execution will be delayed for 10 seconds as it is waiting to get a response from the database, the Web Service, or the Web API. Until Method2 has not completed its execution, Method3 will not be executed because of the sequential execution of the program, i.e., one by one.

Example to Understand the Problem of Single-Threaded Application in C#.

Here, we will not perform any database or Web Service call; instead, we will use the Thread class Sleep method to delay the execution of Method2 for 10 seconds. Following is the signature of the Sleep Method. Another overloaded version of the Sleep method is available, which takes TimeSpan as input, and we will discuss it in our coming articles.

```
public static void Sleep(int millisecondsTimeout);
```

The sleep method takes the time in milliseconds as input and then suspends the current thread execution for that specified number of milliseconds. So, please modify the Program class code as shown below.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    Method1();
    Method2();
    Method3();
    Console.Read();
}
static void Method1()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method1 :" + i);
    }
}
static void Method2()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method2 :" + i);
        if (i == 3)
        {
            Console.WriteLine("Performing the Database Operation Start
//Sleep for 10 seconds
Thread.Sleep(10000);
            Console.WriteLine("Performing the Database Operation Comp]
        }
    }
}
static void Method3()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("Method3 :" + i);
    }
}
}
}

```

Now, run the application and notice that the Method2 execution is delayed for 10 seconds. Once Method2 completes its execution, only Method3 starts its execution. This is because all three methods are executed by a single thread, which is the drawback of the single-threaded application.

How do we solve the above problem?

To solve the above problem, we have a concept called Multithreading in C#. As we already discussed Operating System has Processes used to run our applications. The Process contains Thread, which will actually run our application code.

A process, by default, uses a single thread to run our application code, but a process can have multiple threads, and multiple threads can run our application code simultaneously. In this case, each thread will perform a different task, or you can say each thread will execute different parts of our application code.

In our application, we have three methods. So, the three methods we defined in our application can be executed by three threads. The advantage is that the execution takes place simultaneously. So, when multiple threads try to execute the application code, the operating system allocates some time for each thread.

In our example, we want to execute the three methods using three different threads. let us say t1, t2, and t3. The thread t1 is going to execute Method1, and thread t2 is going to execute the Method2. At the same time, Method3 will be executed by thread t3. Let us modify the Program class as shown below to execute the three methods with three different Threads. Here, the main thread will execute the Main method, and the Main thread will create the child threads.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Thread Started");
            //Creating Threads
            Thread t1 = new Thread(Method1)
            {
                Name = "Thread1"
            };
            Thread t2 = new Thread(Method2)
            {
                Name = "Thread2"
            };
            Thread t3 = new Thread(Method3)
            {
                Name = "Thread3"
            };
            //Executing the methods
            t1.Start();
            t2.Start();
            t3.Start();
            Console.WriteLine("Main Thread Ended");
            Console.Read();
        }
        static void Method1()
        {
            Console.WriteLine("Method1 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
            Console.WriteLine("Method1 Ended using " + Thread.CurrentThread.Name);
        }
        static void Method2()
        {
            Console.WriteLine("Method2 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method2 :" + i);
            }
            Console.WriteLine("Method2 Ended using " + Thread.CurrentThread.Name);
        }
        static void Method3()
        {
            Console.WriteLine("Method3 Started using " + Thread.CurrentThread.Name);
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method3 :" + i);
            }
            Console.WriteLine("Method3 Ended using " + Thread.CurrentThread.Name);
        }
    }
}
```

```

        Console.WriteLine("Method2 Started using " + Thread.CurrentThread.
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Method2 :" + i);
            if (i == 3)
            {
                Console.WriteLine("Performing the Database Operation Start
                //Sleep for 10 seconds
                Thread.Sleep(10000);
                Console.WriteLine("Performing the Database Operation Compl
            }
        }
        Console.WriteLine("Method2 Ended using " + Thread.CurrentThread.Na
    }
    static void Method3()
    {
        Console.WriteLine("Method3 Started using " + Thread.CurrentThread.
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Method3 :" + i);
        }
        Console.WriteLine("Method3 Ended using " + Thread.CurrentThread.Na
    }
}
}
}
}
}

```

As you can see in the above code, we have created three different instances of the Thread class. To the constructor of the Thread class, we need to pass the method name that needs to be executed by that Thread. Then, we call the Start() method on the Thread class, which will start executing the method. You will not get the output sequentially. Run the application and see the output as shown below. The output may vary in your machine.

```

Main Thread Started
Method1 Started using Thread1
Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method2 Started using Thread2
Method2 :1
Method2 :2
Method2 :3
Performing the Database Operation Started
Method1 :5
Method1 Ended using Thread1
Main Thread Ended
Method3 Started using Thread3
Method3 :1
Method3 :2
Method3 :3
Method3 :4
Method3 :5
Method3 Ended using Thread3
Performing the Database Operation Completed
Method2 :4
Method2 :5
Method2 Ended using Thread2

```

Here, the Main thread is going to create all other Threads. Here, the main thread will execute the Main method, and the Main thread will create the child threads called Worker Threads. Now, to see all the threads, run the application in debug mode by putting debugging point and then open **Debug => Windows => Threads** options, then it will open the following window. You can see here that the Main Thread and the Worker threads execute our application code.

Threads					
Search:		X Search Call Stack		Group by: Process ID	
	ID	Managed ID	Category	Name	Location
^ Process ID: 20352 (4 threads)					
▼	18892	1	Main Thread	Main Thread	▼ ThreadingDemo.exe!ThreadingDemo.Program.Main
▼	21204	3	Worker Thread	Thread1	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method1
▼	14468	4	Worker Thread	Thread2	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method2
▼	19304	5	Worker Thread	Thread3	▼ ThreadingDemo.exe!ThreadingDemo.Program.Method3

What is Multithreading in C#?

If multiple threads are used to execute your application code, it is called Multithreading. Multithreading is a mechanism to implement Concurrent Programming where multiple threads operate simultaneously. Threads are lightweight processes that signify the execution path in a program. Thread usage increases the efficiency of an application and reduces CPU cycle time wastage. The main advantage of using Multithreading is the maximum utilization of CPU resources.

Multithreading in C# refers to the ability of the C# programming language and the .NET Framework to create and manage multiple threads of execution within a single process. Threads are lightweight, independent

sequences of instructions that can run concurrently, allowing you to perform multiple tasks simultaneously. Multithreading is a powerful concept in C# and is used to achieve various goals, such as improving application responsiveness, parallelizing tasks, and efficiently utilizing multi-core processors.

Key Features of Multithreading in C#:

- **Thread Class:** C# provides the `System.Threading.Thread` class, which allows you to create and manage threads. Using this class, you can create new threads, start and stop them, set their priorities, and perform various synchronization and coordination tasks.
- **Parallel Programming:** Multithreading is often used for parallel programming, where multiple threads work on separate parts of a problem to improve performance. The .NET Framework provides libraries and constructs like the Task Parallel Library (TPL) and Parallel LINQ (PLINQ) to simplify parallel programming.
- **Improved Responsiveness:** Multithreading can keep the user interface (UI) responsive while performing time-consuming tasks in the background. For example, in a GUI application, you can run a time-consuming computation on a separate thread so that the UI remains interactive and doesn't freeze.
- **Concurrency:** Multithreading enables concurrent execution of code sections, which is useful for scenarios where multiple threads need to access shared resources, perform I/O operations, or handle multiple client requests simultaneously.
- **Thread Safety:** When multiple threads access shared data or resources, it's crucial to ensure thread safety to prevent data corruption and race conditions. C# provides synchronization primitives like locks, mutexes, semaphores, and the Monitor class to help you achieve thread safety.
- **Async/Await:** C# includes the `async` and `await` keywords, simplifying asynchronous programming. These keywords allow you to write asynchronous code that can run concurrently with other tasks without blocking the calling thread. Asynchronous programming is essential for scalable and responsive applications.

Multithreading can greatly enhance the performance and responsiveness of C# applications, but it also introduces complexities related to synchronization, coordination, and potential issues like deadlocks. Therefore, it's essential to use multithreading judiciously and follow best practices to ensure robust and reliable concurrent code.

What is Thread Class in C#?

1. In C#, the Thread class is used to create threads.
2. With the help of the Thread class, we can create **foreground and background threads**. What are the foreground and background threads that we are going to discuss in our coming article?
3. Thread class also allows us to set the priority of a thread. In our coming articles, we will discuss **how to set the priority of a thread**.
4. The Thread class in C# also provides the current state of a thread. We will discuss this when discussing **Thread Life Cycle** in C#.
5. The Thread class in C# is sealed and cannot be inherited.

In the next article, I will discuss **Constructors of Thread Class in C#** with Examples. In this article, I explain the concept of **Multithreading in C#** with Examples. I hope you understood the basics of C# Multithreading with Examples and enjoy this article.